VET-LAB · PROGRAMA 1

Modelos e Moléculas

Guia de instalação, operação e validação do ambiente PostgreSQL 18

Repositório `vetlab-modelos-moleculas` · infraestrutura em `vetlab-infra/` · 30 de outubro de 2026

NESTE GUIA

1. Visão geral
2. Pré-requisitos
3. Estrutura do repositório
4. Subir o ambiente
5. Serviços e portas
6. Credenciais e objetos
7. Operação do dia a dia
8. Validação
9. Observações importantes
10. Problemas comuns

A. Checklist de primeira execução

18.6	6432	9	25/25
PostgreSQL	porta do PgBouncer	extensões analíticas	verificações de saúde

Ambiente local de desenvolvimento
O template publica as senhas em branco e o compose recusa subir até que sejam definidas; o certificado TLS é autoassinado. Não reutilize esta configuração em ambiente compartilhado ou de produção sem distribuir um certificado próprio.

Documento de apoio ao ensaio de caso do Programa 1. Os comandos foram validados no ambiente descrito na seção 8.

1. Visão geral

O ambiente é 100% declarativo: o comando `docker compose up -d --build` cria o cluster PostgreSQL 18, as roles, o banco `vetlab_db`, os schemas de domínio, as extensões analíticas e os grants — sem nenhum passo manual. O pool de conexões (PgBouncer) e a interface de administração (pgAdmin 4) sobem junto.

O que este guia cobre: pré-requisitos, estrutura de pastas, primeira subida, portas e credenciais, rotinas de operação, validação automatizada, observações e solução de problemas comuns.

2. Pré-requisitos

Windows (recomendado)

- **WSL2** com uma distribuição Linux instalada (Debian ou Ubuntu).
- **Docker Desktop** com backend WSL2 e integração habilitada em `Settings › Resources › WSL Integration`.
- **Windows Terminal**. Para executar os scripts `.sh`, use o **Git Bash** ou um terminal WSL com a integração do Docker Desktop ativa.

macOS / Linux

- **Docker Engine** e **Docker Compose v2** instalados.

> **Instalação bare metal não é suportada**
> O bootstrap, as extensões, o TLS e o pool de conexões são provisionados pela imagem Docker. Rodar PostgreSQL direto no host exigiria repetir esse provisionamento manualmente — fora do escopo deste repositório.

3. Estrutura do repositório

Regra de ouro: **o que é "do Docker" mora em** `vetlab-infra/` e o material de leitura mora em `artigos/`. A raiz guarda apenas o README, os arquivos de controle de versão e o acervo.

```
.
├── README.md                  # visão geral, quickstart, operação
├── .editorconfig              # LF, UTF-8, indentação
├── .gitattributes             # normalização de fim de linha
├── .gitignore                 # segredos e dados de runtime
├── artigos/                   # acervo de leitura (PDFs, notas, figuras)
│   ├── pdf/                   # artigos originais
│   ├── notas/                 # fichamentos em Markdown
│   └── figuras/               # imagens extraídas dos PDFs
└── vetlab-infra/              # toda a infraestrutura
    ├── docker-compose.yml     # postgres, pgbouncer, pgadmin
    ├── Dockerfile             # PostgreSQL 18.6 + extensões PGDG
    ├── .dockerignore
    ├── .env / .env.example
    ├── CREDENTIALS.md         # segredos locais (não versionado)
    ├── config/                # postgresql.conf, pg_hba.conf, pgbouncer/, pgadmin/
    ├── docker/                # entrypoint TLS, pg-backup, pg-restore, userlist
    ├── init-scripts/          # bootstrap + sql/00-roles ... 30-grants
    ├── scripts/smoke-test.sh  # 25 verificações de saúde
    ├── docs/                  # arquitetura, ADRs e este guia
    └── data/backups/          # saída do pg-backup
```

> **Por que uma subpasta**
> Isolar a infraestrutura em uma pasta própria mantém o contexto de build mínimo, impede que segredos da raiz entrem na imagem e deixa espaço para a aplicação viver no mesmo repositório depois.

4. Subir o ambiente (primeira vez)

No **PowerShell**, a partir da raiz do repositório:

```
cd D:\source\Academicos-Projetos\vetlab-modelos-moleculas\vetlab-infra

# 1. Variáveis de ambiente
Copy-Item .env.example .env      # Linux/macOS: cp .env.example .env

# 2. Preencha as 4 senhas (o compose não sobe com elas vazias):
#    POSTGRES_PASSWORD, SVC_DB_PASSWORD,
#    SVC_DB_READONLY_PASSWORD, PGADMIN_DEFAULT_PASSWORD
#    openssl rand -base64 24   -> uma por role
notepad .env

# 3. Build + start
docker compose up -d --build

# 4. Status e saúde
docker compose ps
& 'C:\Program Files\Git\bin\bash.exe' ./scripts/smoke-test.sh
```

No primeiro start (volume vazio) o Postgres executa `init-scripts/` e cria roles, banco, schemas e extensões. O certificado TLS autoassinado é gerado automaticamente no volume `pglab_certs`.

> **Primeiro boot do pgAdmin**
> A importação de `servers.json` pode levar de 3 a 8 minutos. Durante esse período a porta 5050 não responde — é normal. Acompanhe com `docker compose logs -f pgadmin`.

5. Serviços e portas

Serviço	Porta no host	Container	Uso
PostgreSQL 18	127.0.0.1:5432	5432	Administração direta (role postgres)
PgBouncer	127.0.0.1:6432	6432	Caminho da aplicação (svc_app, pool transacional)
pgAdmin 4	127.0.0.1:5050	80	Interface web de administração

Todas as portas são configuráveis no `.env` (`POSTGRES_PORT`, `PGBOUNCER_PORT`, `PGADMIN_PORT`). As publicações estão amarradas a `127.0.0.1`: nada fica visível para outras máquinas da rede.

Conexões

```
# Admin (direto no cluster)
psql "host=localhost port=5432 dbname=postgres user=postgres"

# Aplicação (via pooler)
psql "host=localhost port=6432 dbname=vetlab_db user=svc_app"
```

6. Credenciais e objetos do banco

Role	Uso	Acesso
`postgres`	Superusuário / administração	5432 direto
`svc_app`	Aplicação — owner dos schemas	6432 (pooler)
`svc_readonly`	BI e relatórios — apenas SELECT	6432 (pooler)

As credenciais locais ficam em `vetlab-infra/CREDENTIALS.md`, gerado a partir do `.env` e **não versionado**. O template publica as senhas em branco e o compose recusa subir até que sejam definidas — não existe senha padrão.

Objeto	Conteúdo
Banco	vetlab_db (owner svc_app)
Schemas	clinica, genoma, auditoria
search_path	clinica, genoma, auditoria, public
Extensões	vector, pgaudit, pg_cron, pg_partman, hypopg, pg_stat_statements, pgcrypto, pg_trgm, unaccent
TLS	ssl=on; certificado autoassinado no volume pglab_certs
Auditoria	pgaudit registra DDL e role nos logs do container

7. Operação do dia a dia

```
# Logs em tempo real
docker compose logs -f postgres

# psql interativo
docker compose exec postgres psql -U postgres -d vetlab_db

# Backup lógico (valida o dump e aplica a retenção de BACKUP_RETENTION_DAYS)
docker compose exec -u postgres postgres pg-backup vetlab_db

# Restauração (destrutiva; pede confirmação)
docker compose exec -u postgres postgres \
  pg-restore /backups/vetlab_db_<timestamp>.dump vetlab_db

# Reset completo (apaga dados e roda o bootstrap de novo)
docker compose down -v
docker compose up -d
```

Os backups ficam em `vetlab-infra/data/backups/` no host, em formato custom (`pg_dump -Fc --compress=zstd`). A retenção padrão é de 7 dias.

8. Validação

O script `scripts/smoke-test.sh` executa 25 verificações ponta a ponta e termina com código de saída diferente de zero se qualquer uma falhar. Ele roda do host, com o Docker em execução — no Windows, use o Git Bash:

```
& 'C:\Program Files\Git\bin\bash.exe' ./scripts/smoke-test.sh

# Resultado esperado
#   Resultado: 25 PASS / 0 FAIL
```

O que é verificado:

- os três containers em execução (postgres, pgbouncer, pgadmin);
- conexão aceita e TLS ativo (`ssl=on`);
- schemas `clinica`, `genoma` e `auditoria`;
- as 9 extensões instaladas;
- roles `svc_app` e `svc_readonly` e o `search_path` do banco;
- login das duas roles pelo pooler e bloqueio de DDL para a role somente leitura;
- pgAdmin respondendo HTTP (qualquer código abaixo de 500);
- geração e validação de um dump de backup.

> **Exit code no PowerShell**
> O `pg-backup` escreve o progresso em stderr, e o PowerShell 5.1 exibe isso como *NativeCommandError*. Não é falha: confira a última linha (`Resultado: 25 PASS / 0 FAIL`). Para checar o código de saída de verdade: `bash -c './scripts/smoke-test.sh >/dev/null 2>&1; echo exit=$?'`.

9. Observações importantes

- **Portas ocupadas.** Se aparecer `port is already allocated`, outra instância usa a porta. Ajuste `POSTGRES_PORT`, `PGBOUNCER_PORT` ou `PGADMIN_PORT` no `.env`.
- **init-scripts rodam uma única vez.** Eles só executam com o volume vazio. Para aplicá-los de novo, use `docker compose down -v` (isso apaga o cluster).
- **TLS autoassinado.** `sslmode=require` funciona; `verify-full` exige distribuir o `server.crt` gerado no volume `pglab_certs`.
- **Autenticação.** O TCP exige `scram-sha-256`; `trust` vale apenas para o socket local dentro do container.
- **pgAdmin em modo desktop.** Não há master password — adequado somente para uso local.
- **Pool transacional.** O modo `transaction` não suporta `PREPARE`, `LISTEN/NOTIFY` nem cursores nomeados entre transações. Quem precisar disso conecta direto na 5432.
- **pg_cron restrito ao banco principal.** Restaurar um dump em um banco com nome diferente de `vetlab_db` gera um erro esperado (`can only create extension in database vetlab_db`); o restante da restauração é aplicado normalmente.
- **Sem replicação nem PITR.** O backup é lógico (`pg_dump`), então o RPO equivale ao último backup. Para recuperação point-in-time seria necessário WAL archiving.
- **Scripts .sh no Windows.** O `bash` do WSL não enxerga o Docker Desktop sem a integração habilitada. Use o Git Bash (`C:\Program Files\Git\bin\bash.exe`).

- **Fim de linha.** O `.gitattributes` força LF em `.sh`, `.sql` e `.conf`. Se um arquivo vier de fora do git, converta para LF antes de rodar — CRLF quebra o entrypoint e os scripts.
- **Repositório público, segredos fora dele.** O projeto é aberto no GitHub, mas `.env` e `CREDENTIALS.md` estão no `.gitignore` — nenhum clone traz senha pronta.
- **Portas só em localhost.** As publicações usam `127.0.0.1`; para expor de propósito, remova esse prefixo no `docker-compose.yml` e assuma o risco.

10. Problemas comuns

Sintoma	Causa / solução
port is already allocated	Outra instância usa a porta. Ajuste as variáveis de porta no .env.
database "vetlab_db" does not exist	Mensagem transitória do pg_cron durante o bootstrap; desaparece quando o banco é criado.
init-scripts não rodaram de novo	Eles só executam com volume vazio. Use docker compose down -v para recriar.
Erro de CRLF nos scripts	O .gitattributes força LF; converta arquivos vindos de fora do git.
docker: command not found no smoke test	Terminal WSL sem integração com o Docker Desktop; use o Git Bash.
pgAdmin não abre / erro 401 em /login	Primeiro start lento, ou requisição sem token CSRF. Aguarde e acesse pelo navegador.
Porta 5050 recusa conexão por vários minutos	Importação inicial do servers.json; acompanhe docker compose logs -f pgadmin.

Referências no repositório: `README.md`, `vetlab-infra/docs/ARCHITECTURE.md` e `vetlab-infra/docs/decisions/` (ADRs 0001 a 0004).

Apêndice — checklist de primeira execução

Marque cada item depois de subir o ambiente pela primeira vez em uma máquina nova.

- [ ] Copiei `.env.example` para `.env` e revisei as senhas.
- [ ] `docker compose config` validou o arquivo sem erros.
- [ ] `docker compose up -d --build` terminou com os três serviços em execução.
- [ ] `docker compose ps` mostra postgres e pgbouncer como *healthy*.
- [ ] `scripts/smoke-test.sh` retornou 25 PASS / 0 FAIL.
- [ ] Acessei o pgAdmin em `http://localhost:5050` e os dois servidores aparecem.
- [ ] `pg-backup` gerou um dump em `data/backups/`.
- [ ] Testei um `pg-restore` em um banco descartável.
- [ ] Registrei as portas efetivas e as credenciais em `CREDENTIALS.md`.